



Project Report

Centralized Incident Management Portal

Prepared by: Agaba Joel Muhanguzi

Introduction:

The Incident management portal is a high performance web application that is designed to ease IT support workflows across bank branches. In a fast paced banking environment , hardware issues, system glitches and network failures must be logged, prioritized and resolved as quick as possible

This portal replaces the traditional and manual methods with a new database backed queue system. Branch Tellers can instantly report incidents, while IT Support technicians can dynamically assign, track and resolve tickets in real time. I built this system on the modern PHP framework called Laravel and the system is designed for maximum speed, implementing relational database integrity and robust security.

Learning Journey

A defining aspect of this project is the rapid technical upscaling required to build it. At the start of the activity, I had no prior experience with PHP, relational database migrations as well as applying modern Model View Controller software architecture.

Through this i managed to learn and apply several industry standard concepts like;

1. Transitioning to backend frameworks as I was used to static frontend design and now to dynamic server side development using PHP.
2. I learnt database migrations which also included designing, altering and version control database schemes directly in PHP code, completely bypassing manual error prone database management tool adjustments.

3. I learnt trouble shooting and went deeper into DevOps literacy as I acquired critical command line debugging skills, successfully navigating local environments, resolving git-branch synchronization conflicts and finally pushing clean code to GitHub.

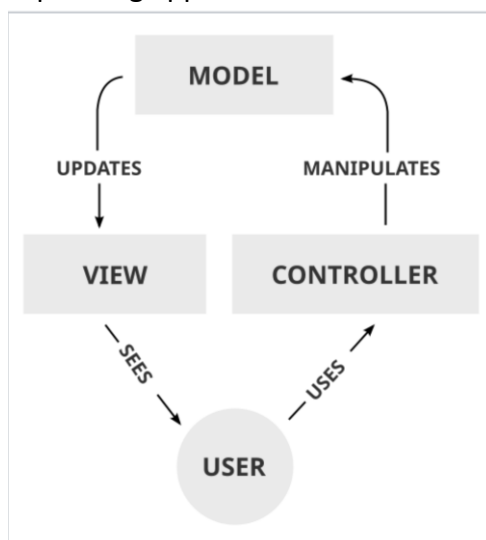
Core Objectives of the Project

In order for me to prove the system is viable for Pride Bank, i built the portal to meet 4 operational standards as seen below;

- **Strict Data Validation:** Ensuring all incoming tickets are automatically sanitized and verified before touching the database to prevent injection attacks or broken data.
- **Concurrency Protection:** Implementing atomic locking to prevent race conditions such as two technicians accidentally modifying or overriding the same ticket at the exact same millisecond.
- **Background Processing:** Offloading heavy tasks to background queue workers, ensuring a fast user experience
- **Auditability and Accountability:** Automatically linking every ticket to its respective creator and technical assignee via database constraints.

System Architecture

Laravel uses an MVC pattern and this keeps our code clean, organized and scalable by separating applications into 3 distinct layers.



The components

- The route (web.php): This is the entry point and It captures incoming web URLs and sends them to the correct Controller.
- The controller (TicketController.php): This can be thought of as the brain of the system. It contains the business logic. It receives requests, fetches data, applies rules and passes information along.

- The model (Ticket.php, User.php): This is the data manager of the system and It communicates directly with database tables using PHP, eliminating the need to write raw SQL.
- The view (.blade.php): This is simply the presentation layer. It takes the data from the controller and formats it into clean HTML for the user's browser.

Database Schema & Relational Design

Instead of manually building tables, I designed our entire database schema using Laravel Migrations. This ensures the structure is perfectly tracked via version control.

Primary Application Tables

Table Name	Primary Purpose	Key fields
Users	Tracks accounts and access levels	id, email (unique), password, role
Tickets	Core incident levels	title, description, category, priority, status, branch_location

System Architecture Tables

Table Name	Role	Technical Impact
Cache and cache locks	Performance and concurrency	Caches frequent queries to speed up loading.
Jobs and failed jobs	Asynchronous Processing	Offloads slow to a background queue so the browser never freezes.

Implementation of relations via foreign keys

- user_id → users: Links the ticket directly to the reporting Teller.
- assigned_to → users.id: Links the ticket to the assigned IT Technician.

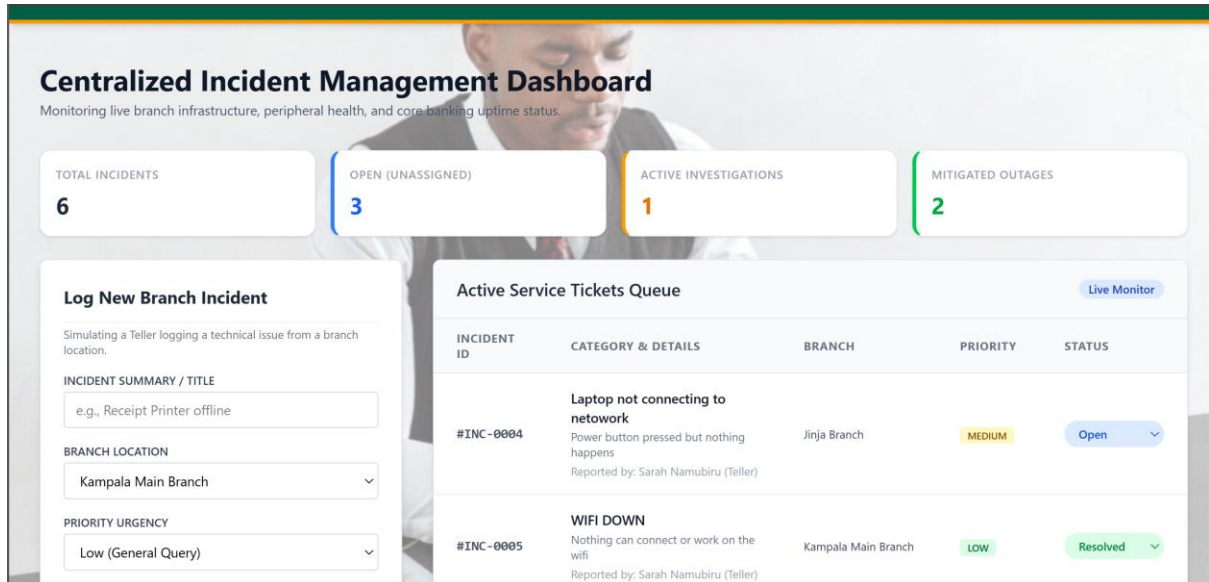
Controller operations

We structured the backend logic into 3 operations as seen below;

- Dashboard Reading: This fetches tickets and their relationships in a single database query hence prevents performance lag and all metrics on the dashboard are computed dynamically.

- **Data Entry Validation:** Enforces server side validation rules. It rejects empty inputs, limits text length and restricts the priority field strictly to; low, medium, high, critical.
- **Ticket Lifecycles:** Automatically assigns an available IT Support user to the ticket as soon as its status moves from "Open" to "In Progress" or "Resolved". If moved back to "Open", it clears the assignee automatically.

Dashboard



Future development and implementation

The project has been uploaded to github at <https://github.com/JOELAGABA/Ticket-reporting-system-for-bank-IT-support-environment-> and the intention is to keep building up this project with new information, standards and techniques that i will learn along the way.

Conclusion

This project has introduced me to the field of webapp development and skills that i can apply in the future to solve a variety of problems just like the one handled today. There's still much to learn and build on but as a mock project, this has been a beneficial exercise.